

Problem 1

DFS 와 BFS 의 장/단점 또는 특징

DFS 는 한 방향으로 끝까지 탐색한 후 더 이상 갈 수 없게 되면 가장 가까운 갈림길로 되돌아와 다른 방향을 탐색하는 방식이며, 너비 우선 탐색 BFS 는 시작 정점에서 가까운 정점을 우선 방문하고 멀리 있는 정점을 나중에 방문하는 방식입니다. DFS 는 스택이나 재귀 함수를 활용하여 자료구조에 마지막으로 추가된 노드를 우선적으로 방문하는 반면, BFS 는 큐를 활용하여 먼저 저장된 노드를 우선적으로 방문한다는 구조적 차이가 있습니다. 두 가지 탐색 모두 인접 행렬로 구현할 경우 $O(n^2)$ 의 시간이 소요되며, 인접 리스트로 구현할 경우 $O(n + e)$, (e 는 간선의 개수)의 시간이 소요되므로 조밀 그래프에서는 인접 행렬을, 희소 그래프에서는 인접 리스트를 사용하는 것이 유리하다는 특징을 갖습니다.

인터넷을 더 찾아본 결과 두 알고리즘의 연산 효율성은 트리나 그래프의 모든 정점과 간선을 탐색하므로 기본적으로 동일하지만 메모리 효율성 면에서 조금 다르다는 것을 발견했습니다.

DFS 는 탐색하는 경로를 따라 깊이 들어갈 때 필요한 노드들만 스택에 저장하므로 적은 메모리만 차지하여 메모리 효율성이 좋은 반면 BFS 는 같은 깊이에 있는 모든 인접 노드를 큐에 저장한 뒤 다음 단계로 넘어가야 하므로, 그래프의 너비가 넓을수록 메모리 소모가 급격하게 증가한다는 문제가 있습니다. 따라서 최단 경로를 보장해야 한다면 BFS 를 사용하는 것이 좋지만, 메모리 제약이 크거나 목표가 트리 깊은 곳에 있을 확률이 높다면 DFS 를 사용하는 것이 더 적합할 것이라고 생각합니다.

Problem 2

왜, 어떻게 설계했는지

2차원 배열 격자 안에서 서로 상하좌우로 연결된 빈 공간들의 덩어리 개수를 세는 문제이므로, 하나의 빈 공간에서 출발해 연결된 모든 빈 공간을 방문 처리하여 하나의 독립된 무대로 묶어주기 위해 BFS 를 구성했습니다.

방문 여부를 기록할 2차원 리스트 visited를 생성한 뒤 격자의 모든 칸을 이중 반복문으로 순회하면서 무대공간임과 동시에 아직 방문하지 않은 칸을 발견하면 이를 새로운 무대의 시작점으로 판단하여 BFS 탐색을 진행합니다.

이후 상, 하, 좌, 우로 연결된 모든 0들을 Queue에 넣어 visited 처리했습니다.

하나의 BFS 탐색이 끝나면 연결된 한 덩어리의 무대 공간을 모두 찾은 것이 되기 때문에 answer를 1 증가시키는 로직으로 구성해봤습니다,